

Raspberry Pi 上の Embedded Xinu のマルチコア化

ワーカープール型 SMP の設計と実測

何が速くなり、何が速くならないか

児玉靖司

法政大学経営学部

yass@hosei.ac.jp

2026 年 7 月 16 日

Abstract

Raspberry Pi 上でベアメタル動作する Embedded Xinu を、搭載された 4 つの CPU コアを使うように拡張した。設計は、コア 0 が既存の単一コア OS（スケジューラ・アクター・ネットワーク・ウィンドウシステム）を一切変更せずに実行し続け、残るコアを純計算専用のワーカーとして起こす「ワーカープール型」である。対称型 SMP を採らないのは、既存ランタイムが非リエントラントであり、共有 ready キューが実績のある機能を破壊するためである。

本レポートの主眼は、速くなったという報告ではなく、**何が速くなり、何が速くならないかを実測で切り分けた点にある**。計算律速の処理は最大 $\times 3.99$ （ほぼ理想値）まで向上した一方、メモリ書込律速の処理は $\times 0.95$ – $\times 1.35$ に留まり、並列化の意味がないことが分かった。この結果は「ウィンドウシステムを 4 コアで速くできるか」という問いに否と答える根拠となった。またワーカープールの起動コストは実測 7–11 μs であり、これを下回る仕事は並列化するとかえって遅くなる（実測 $\times 0.38$ ）。この 2 つの境界が、並列化すべき対象を決める。

Raspberry Pi 3 についての実装と測定を第 4 節に示す。Raspberry Pi 4 および 5 についても同一の枠組みで追記予定である。

Contents

1 はじめに	3
2 共通設計：ワーカープール型 SMP	3
2.1 なぜ対称型 SMP を採らないか	3
2.2 ロックフリー調停が成立する理由	3
2.3 ジョブ引数は呼び出し側スタックに置く	3
2.4 プールの排他とフォールバック	4
3 測定方法	4
3.1 時計：割り込みではなくフリーランニングカウンタを読む	4
3.2 正しさの同時検証	4
3.3 絶対時間ではなく比を報告する	4
4 Raspberry Pi 3 (BCM2837)	4
4.1 対象と前提	4
4.2 コア起動：Pi 3 固有の差分	5
4.3 実測 1：計算律速の処理は 4 コアでスケールする	5
4.4 実測 2：メモリ律速の処理はスケールしない	6
4.5 実測 3：プールの往復コストと損益分岐点	6
4.6 適用判断 1：ウィンドウシステムは並列化しない	7
4.7 適用判断 2：AIPL アクターは 1 箇所だけが該当する	7
4.8 発見した不具合	8
4.9 副次的な発見：volatile グローバルの代償	8

5	ボード横断比較	8
6	結論	9

1 はじめに

Raspberry Pi に搭載される BCM2837 / BCM2711 / BCM2712 は、いずれも 4 コアの ARM プロセッサを持つ。しかし Embedded Xinu は単一コア OS であり、これらのボード上では 1 コアしか使わず、残る 3 コアは起動されないまま放置されていた。本作業の目的は、この遊休コアを実際の計算に使うことである。

本レポートは次の順で述べる。第 2 節で全ボード共通の設計方針を、第 3 節で測定方法を述べる。第 4 節以降がボード別の実装と実測である。第 5 節で横断比較を行い、第 6 節で結論を述べる。

2 共通設計：ワーカープール型 SMP

2.1 なぜ対称型 SMP を採らないか

素朴な発想は、ready キューを全コアで共有する対称型 SMP である。しかし既存の Xinu ランタイム (AIPL アクター、ネットワークスタック、USB、ウィンドウマネージャ) は非リエントラントであり、複数コアから同時に呼ばれることを想定していない。共有 ready キューを導入すれば、実機で動作実績のあるこれらの機能を全面的に作り直す必要が生じ、リスクが利得を上回る。

そこで本設計では次の非対称構成を採る。

- ・ **コア 0** は既存の単一コア OS をそのまま実行する。ready キューはコア 0 の専有であり、スケジューラ・アクター・ネットワーク・描画は一切変更しない。
- ・ **コア 1-3** は専用の計算ワーカーとして起動する。低消費電力の WFE で待機し、投入されたジョブ (仕事の範囲) を実行して完了を通知する。OS には触れず、割り込みも受けず (IRQ/FIQ マスク)、メモリ確保も行わないため、カーネルと競合し得ない。

この構成は、既存機能への退行リスクが構造的にゼロであるという点で優れている。コア 0 の挙動が変わらないからである。

2.2 ロックフリー調停が成立する理由

ワーカーとコア 0 の調停は、単一のジョブメールボックス (volatile な配列群) と dsb バリアのみで行い、スピンロックも LDREX/STREX も使わない。これが成立するのは、この移植が **D-cache を OFF にしている**ためである。すなわち全てのデータアクセスが RAM に直行するので、コア間のキャッシュコヒーレンシ問題が原理的に生じない。

これは設計上の仮定であり、実機で検証した (第 4 節)。なお D-cache を有効化すると、この仮定は崩れ、ジョブメールボックスには実際のキャッシュ管理が必要になる。

2.3 ジョブ引数は呼び出し側スタックに置く

ジョブの引数をファイルスコープの変数で渡してはならない。コア 0 はプリエンプティブであり、かつプールの利用者は複数スレッド (HTTP サーバスレッドと AIPL アクタースレッド) に及ぶ。ゆえに次の競合が起こり得る。

1. スレッド A が引数を書く。
2. A がプリエンプトされる。
3. スレッド B が引数を上書きし、プールを取って実行する。
4. A が再開し、B の引数で計算して誤った答えを返す。

本設計では API に不透明ポインタ `ud` を設け、引数を呼び出し側のスタックに置くことで、この競合を構造的に起こり得なくした。ロックで塞ぐより安全であり、副次的に性能も向上した (第 4.9 項)。

2.4 プールの排他とフォールバック

ジョブメールボックスは 1 組しかないので、プールは同時に 1 人しか使えない。既に使用中の場合、後続の呼び出し元は自分の全チャンクをその場で実行する。待たせないでデッドロックせず、並列化の利得を諦めるだけで答えは常に正しい。

同様に、ワーカーが応答しない場合はコア 0 がそのチャンクを引き取る。したがってワーカーの不調が OS を停止させることはない。

3 測定方法

3.1 時計：割り込みではなくフリーランニングカウンタを読む

当初、経過時間の測定には Xinu の `clktime/clkticks` を用いていたが、どの測定も 0ms を返した。原因はこれらがタイマ割り込みで加算される変数であり、その割り込みが安定して発火していないことにある。これはマルチコア化以前のカーネルでも同様であり、本作業が持ち込んだ問題ではない。

そこで System Timer の 1MHz フリーランニングカウンタを直接読む `clkcount()` に切り替えた。割り込みに依存しないため停止せず、分解能も 1ms から 1μs へ 1000 倍向上した。

3.2 正しさの同時検証

全てのベンチマークは、同一の仕事を逐次で 1 回、並列で 1 回実行し、両者の答えが一致するかを `agree` として毎回報告する。速いが誤った結果を、速いという理由で見逃さないためである。本レポートに載せた全測定で `agree = yes` である。

さらに、分割ロジック（剰余類による仕事の配分）は実機に投入する前に、カーネルのソースから当該関数をそのまま抜き出してホスト上で総当たり検証した。この検証により、実機に載せる前に実際に 2 件の誤りを発見・修正している（第 4.8 項）。

3.3 絶対時間ではなく比を報告する

測定中に、CPU クロックが一定でないことを発見した。再起動直後の最初の測定だけが速く、以降は安定した遅い値に落ち着く。その比は実測 2.33 で、Raspberry Pi 3 B+ の $1.4\text{GHz} \div 600\text{MHz} = 2.33$ と一致する。ベアメタル Xinu は DVFS を制御しないため、ファームウェアがクロックを変更しているものと考えられる。

したがって絶対時間は再起動からの経過に依存し、比較に使えない。一方 速度向上比 は安定している。1 コア測定と 4 コア測定を同一リクエスト内で連続実行しているため、クロックの影響が相殺されるからである。実際、繰り返し測定で比は完全に一致した（例：×3.99 が 3 回とも同値）。本レポートは比を主たる指標とし、絶対時間は状態を明記した上で参考値として示す。

4 Raspberry Pi 3 (BCM2837)

4.1 対象と前提

対象は Raspberry Pi 3 B+ (BCM2837、Cortex-A53 × 4、1.4GHz) 上の Embedded Xinu (リポジトリ `xinu-raz/xinu`) である。このボードは WiFi・有線 LAN・HDMI ウィンドウマネージャ・AIPL アクターランタイム・JIT が実機で動作している一方、`include/rcu.h` に「*The arm-rpi3 port parks the secondary cores (MPIDR guard)*」とあるとおり、4 コアのうち 3 コアを起動せずに放置していた。

本移植の前提として重要なのは、このポートの MMU 設定である。`system/platforms/arm-rpi3/mmu.c` は identity map (VA = PA) を張り、MMU と I-cache は有効、D-cache は無効のままにしている。実機で読み出した SCTLR がこれを裏付ける。

```
enabled=1
sctlr=0x00c51839 M=1 C=0 I=1 Z=1
ttbr0=0x0011c000
```

Listing 1: 実機 /api/mmu の応答 (D-cache OFF の確認)

$C = 0$ 、すなわち D-cache は OFF である。よって第2節のロックフリー調停の前提が、この実機で成立していることが確認できた。

4.2 コア起動：Pi 3 固有の差分

Pi 4 / Pi 5 は AArch64 で PSCI やスピンテーブルによりコアを起動するが、Pi 3 は 32bit (`arm_64bit=0`) で動作しており、機構が異なる。32bit のファームウェアはコア 0 しか起動せず、コア 1-3 は `armstub7` の中で自分専用のメールボックス 3 を監視して待機している。そこへ入口アドレスを書き込み、`sev` で起こす。

```
/* ARM-local per-core mailbox 3 SET register. The 32-bit Pi 3 firmware leaves
 * cores 1-3 spinning in armstub7 on exactly this word; writing a jump target
 * and issuing SEV releases the core to that address. This MMIO page is
 * already Device-mapped by mmu.c (0x40000000..0x40FFFFFF). */
#define ARM_LOCAL_MBOX3_SET(core) \
    ((volatile unsigned long *) (0x4000008CUL + 0x10UL * (unsigned long)(core)))
```

Listing 2: ARM ローカルのコア別メールボックス 3 (`system/platforms/arm-rpi3/smp.c`)

この MMIO 領域は既に `mmu.c` が Device 属性で写像済みであったため、追加の写像は不要であった。起こされたコアは `start.S` に追加した `_smp_start` トランポリンに入り、SVC モードへ正規化し、IRQ/FIQ をマスクし、コア専用スタックを設定して C の入口へ入る。そこでコア 0 と同一の MMU 設定 (コア 0 が構築した L1 テーブルを共有し、MMU と I-cache を有効化、D-cache は無効のまま) を適用する。設定を揃えるのは体裁の問題ではなく、4 コアの時間比較を公平にするためである。

なお、コア解放の手掛かりを 2 系統用意した。`armstub7` のメールボックスと、`start.S` 側の `smp_release[]` である。後者はファームウェアが 4 コアとも `_start` に流し込む実装だった場合の保険である。この配列は意図的に `.bss` ではなく `.data` に置いた。コア 0 が `.bss` をゼロクリアしている最中に、待機中のコアが古い値を読むことを防ぐためである。

Table 1: 起動が正しく構成されたことの確認 (リンク結果)

シンボル	アドレス	セクション
<code>_smp_start</code>	<code>0x00008280</code>	<code>.text</code>
<code>smp_release</code>	<code>0x0010e980</code>	<code>.data (D)</code>
<code>smp_stacktop</code>	<code>0x0010e990</code>	<code>.data (D)</code>

実機での結果

最大の不確実性は「32bit ファームウェアが本当にこのメールボックスでコアを解放するか」であった。実機で確認された。

```
SMP bench kind=nqueens
cores_online = 4
```

Listing 3: /bench の応答 (4 コアが起動している)

仮に外れていても、起動待ちは短くバウンドしてあり、応答しないコアは offline 扱いで 1 コアにフォールバックして起動は継続する設計とした。

4.3 実測 1：計算律速の処理は 4 コアでスケールする

表 2 に計算律速のベンチマークを示す。値は全て安定状態 (再起動後、クロックが落ち着いた状態、アクター未起動) での測定であり、第3.3項のとおり比を主指標とする。

Table 2: Pi 3：計算律速の速度向上 (/bench、cores_online = 4)

ベンチマーク	性質	1 コア (μ s)	4 コア (μ s)	速度向上	agree
dining n=5	純レジスタ演算	119,626	29,920	$\times 3.99$	yes
nqueens n=12	純計算 (再帰)	260,053	70,149	$\times 3.70$	yes
nqueens n=11	純計算 (再帰)	50,803	15,022	$\times 3.38$	yes
nqueens n=13	純計算 (再帰)	1,419,536	445,639	$\times 3.18$	yes
primes n=200000	計算 (下記参照)	155,686	75,958	$\times 2.04$	yes

dining が理想値 $\times 4.00$ にほぼ届く $\times 3.99$ を示したことは、本設計の妥当性を端的に示している。この処理はメモリに一切触れない純粋なレジスタ演算であり、コア間で奪い合う資源が存在しないためである。

primes だけが $\times 2.04$ と低い。これは偶然ではなく、仕事の配分がこの問題の構造と共鳴した結果である。スライド 4 で配ると、コア 0 とコア 2 は「4 の倍数」と「 $4n+2$ 」すなわち全て偶数を担当し、 $d=2$ で即座に合成数と判明して一瞬で終わる。一方コア 1 と 3 が奇数を全て引き受ける。つまり実質 2 コアしか働いておらず、それがそのまま $\times 2.04$ という数字になっている。N-Queens では同じスライド配分が正しく機能している (コストが列位置に対してなめらかで、偶奇と相関しないため)。ブロック巡回配分に変えれば解消するが、これはベンチマークの均衡の問題であり、本レポートの結論には影響しない。

4.4 実測 2：メモリ律速の処理はスケールしない

表 3 に、大量のメモリ書込 (fill) の測定を示す。これは描画 (フレームバッファへの書込) と同じ性質の処理であり、ウィンドウシステムを並列化すべきかを判定するために用意した。

Table 3: Pi 3：メモリ書込律速の速度向上 (fill、8 パス)

語数	1 コア (μ s)	4 コア (μ s)	速度向上
64	5	13	$\times 0.38$
256	11	18	$\times 0.61$
1,024	42	37	$\times 1.13$
4,096	174	136	$\times 1.27$
16,384	689	511	$\times 1.34$
65,536	2,665	2,137	$\times 1.24$
262,144 (=1 MB)	11,007	8,125	$\times 1.35$

2 つの重要な事実が読み取れる。

第一に、**どれだけ仕事を大きくしても $\times 1.35$ 程度で頭打ちになる**。1 MB (1280 $\times$ 800 の画面の約 1/4 に相当) を書いても $\times 1.35$ であり、計算律速の $\times 3.99$ とは比較にならない。なお高クロック状態で測ると同じ 1 MB が $\times 0.95$ 、すなわち **4 コアの方がわずかに遅い**。いずれの状態でも、並列化に意味がないという結論は変わらない。

第二に、**小さい仕事は並列化すると遅くなる**。64 語では $\times 0.38$ 、すなわち 2.6 倍遅い。これはプールの往復コストに負けているためである。

4.5 実測 3：プールの往復コストと損益分岐点

何もしないジョブを投入して、プール自体の往復コストを測定した (null)。結果は **7–11 μ s** であった。

これが並列化の**損益分岐点**である。すなわち、これを下回る仕事をワーカーに投げると必ず損をする。効率よく利得を得るには、その 10 倍、すなわち **1 メッセージあたり 70 μ s 以上の純計算が必要**となる。この数値が、以降の適用判断の基準となった。

4.6 適用判断 1: ウィンドウシステムは並列化しない

「ウィンドウシステムなどのアプリケーションが 4 コアで速くなるか」という問いに対し、上記の測定は否と答える。理由は 2 つあり、どちらか一方だけでも致命的である。

1. **描画はメモリ律速である。**このポートは D-cache が OFF なので、描画とはフレームバッファへの書込そのものであり、第4.4項の fill と同じ性質を持つ。測定値は $\times 0.95 - \times 1.35$ であり、コアを足しても速くならない。
2. **粒度が足りない。**ウィンドウシステムの描画 1 回は小さい。たとえばカーソルは $12 \times 12 = 144$ 画素であり、往復コスト $7-11 \mu\text{s}$ を下回る。実測でも 64 語の仕事は $\times 0.38$ と遅くなっている。並列化すれば体感はむしろ悪化する。

したがってウィンドウシステムには手を加えなかった。速くならない上に遅くなる変更を入れないことも、工学上の判断である。このボードにおける法則は「計算はスケールする、描画はスケールしない」と要約できる。

4.7 適用判断 2: AIPL アクターは 1 箇所だけが該当する

Pi 3 の AIPL ランタイムは、アクター 1 体につき Xinu スレッド 1 本とメールボックス（リングバッファ+計数セマフォ）を持ち、スレッドがメッセージを取り出して dispatch() でメソッド本体を実行するモデルである。

損益分岐点 $70 \mu\text{s}$ に照らすと、ほとんどの AIPL メソッドは対象外である。bump や ping-pong は算術数命令と send で構成され、基準を 2-3 桁下回る。加えて send・print・wait といったカーネル呼び出しは、そもそもワーカーコアで実行できない。

唯一の例外がロードバランサの Worker.compute_nq である。

```
int ncores = smp_cores_online();
unsigned long t0 = clkcount();
long result = (long)abcl_nq_count_partial_smp((int)n, (int)first_col, ncores);
long elapsed = (long)((clkcount() - t0) / 1000UL); /* us -> ms */
...
enqueue(self_id, disp, "done", 3, ...); /* kernel call: stays on core 0 */
```

Listing 4: 対象となる構造 (apps/abcl_program.c)

重い計算が abcl_nq_count_partial() の一点に隔離され、その前後がカーネル呼び出しという理想的な構造である。n=13 で 1 メッセージあたり約 100 ms の純計算であり、基準を 3 桁上回る。ここだけを 4 コア化し、カーネル呼び出しはコア 0 に残した。

分割の方法

first_col 1 つ分は 1 回の再帰呼び出しであるため、分割は 1 段下、すなわち 2 行目の合法な列で行う。各コアにその剰余類を配る。連続分割ではなくインターリーブとしたのは、コストが列位置に対して中央が重く左右対称であり、連続分割では中央のコアに仕事が偏るためである。

各コアは専用の盤面配列を持つ必要がある。nq_recurse は探索の下降中に cols[] を破壊的に更新するため、配列を共有するとコア同士が互いの探索を壊すからである。この配列を静的変数ではなく呼び出し側スタックに置いたのは、プールが埋まって inline 実行に落ちたスレッドと、プールを保持するスレッドとが、どちらもコア 0 上で同じ配列を奪い合うためである。

ホスト上での総当たり検証により、この分割の正しさとバランスを確認した。

```
REAL-SOURCE partial n=1..12 x every first_col x nc=1..4: ok
REAL-SOURCE summed partials vs known N-Queens counts: ok

balance n=13 first_col=6 nc=4 (per-core counts):  2233  2233  1802  1802
                                                    max/avg = 1.11
```

Listing 5: カーネルのソースから当該関数を抜き出したホスト検証

実機での結果

アクター経路 (/api/loadbal/nqueens?n=12) は正しく完走した。

```
nqueens n=12 submitted=12 task_id_range=1..12
nqueens-result done=1 received=12 expected=12 total=14200
```

Listing 6: アクター経由の N-Queens (n=12 の正解は 14200)

答えは正しく、また各 Worker の busy_ms に値が入るようになった (従来は壊れた clkticks を使っていたため常に 0 であった)。

4.8 発見した不具合

本作業では、実機に投入する前のホスト検証と、測定データの精査により、以下の不具合を発見・修正した。いずれも測定していなければ気付かなかったものである。

1. 素数分割の剰余類の誤り (自作、投入前に発見)。開始値の算出を誤り、剰余類がずれて一部の値を取りこぼし、かつストライド 1 のとき 1 を素数に数えていた。ホスト検証で捕捉。
2. ジョブ引数のスレッド間競合 (自作、投入前に発見)。第2.3項に述べた競合。アクターは Worker が 4 体、すなわちスレッド 4 本であり、実際に起こり得た。API に ud を追加して構造的に解消した。
3. smp_parallel_sum の再入不可 (自作、投入前に発見)。単一のジョブメールボックスを複数スレッドが同時に使うと破壊し合う。使用中なら呼び出し側で全チャンクを実行する方式で解消した。
4. タイマ割り込み由来の時計が常に 0 (既存)。第3.1項。本作業以前から存在した。clkcount() 直読で回避した。

4.9 副次的な発見: volatile グローバルの代償

第2.3項の競合対策として、ジョブ引数を volatile なファイルスコープ変数から呼び出し側スタックの構造体へ移したところ、**速度向上比が ×3.04 から ×3.70 へ改善した**。

理由は D-cache が OFF であることにある。volatile 変数は参照のたびに必ず実メモリを読むため、D-cache のないこの環境では内側ループの 1 反復ごとに RAM 往復が発生していた。通常の構造体メンバに変えたことでコンパイラがレジスタに載せられるようになった。

すなわち競合バグの修正が、同時に性能改善でもあった。D-cache を持たない環境では、volatile の乱用は通常環境より遥かに高くつく。

5 ボード横断比較

Table 4: ボード別ワーカープール型 SMP の比較 (rpi4 / rpi5 は追記予定)

	Pi 3 (BCM2837)	Pi 4 (BCM2711)	Pi 5 (BCM2712)	備考
CPU	Cortex-A53 ×4	Cortex-A72 ×4	Cortex-A76 ×4	
実行状態	AArch32 (32bit)	AArch64	AArch64	Pi 3 のみ 32bit
コア解放	armstub7 mailbox 3	spin table	PSCI	第4.2項
D-cache	OFF	OFF	OFF	ロックフリーの前提
最大速度向上	×3.99	—	—	計算律速
プール往復	7–11 μs	—	—	並列化の損益分岐

Raspberry Pi 4 および 5 についても、本レポートと同一の枠組み (同一の /bench ルート、同一の agree 検証、同一の比による報告) で測定し、本節の表と各ボード節に追記する予定である。

6 結論

1. 遊休していた 3 コアを、既存 OS を一切変更せずに計算へ動員できた。計算律速の処理で最大 $\times 3.99$ (理想値 $\times 4.00$) を実測した。
2. **何でも速くなるわけではない**。メモリ書込律速の処理は $\times 0.95$ – $\times 1.35$ に留まる。またプールの往復コスト $7\text{--}11\mu\text{s}$ を下回る小さな仕事は、並列化すると**遅くなる** (実測 $\times 0.38$)。この 2 つの境界が適用対象を決める。
3. 上記の測定に基づき、ウィンドウシステムは並列化しないという判断を下した。描画はメモリ律速であり、かつ 1 回の描画が往復コストを下回るため、速くならないばかりか遅くなるからである。速くならない変更を入れないことも、工学上の成果である。
4. AIPL アクターについては、判定基準 (1 メッセージあたり $70\mu\text{s}$ 以上の純計算を持ち、`send/print/wait` を含まない) を満たす唯一の対象に適用した。
5. 測定基盤そのものにも 2 つの落とし穴があった。タイマ割り込みに依存した時計が常に 0 を返していたこと (第3.1項)、および CPU クロックが一定でないこと (第3.3項) である。いずれも、比を指標とし、フリーランニングカウンタを直読することで回避した。